

Министерство науки высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)

Факультет информационных технологий и программирования

Образовательная программа Разработка программного обеспечения / Software Engineering

Направление подготовки (специальность) 09.03.02 - Информационные системы и технологии

О Т Ч Е Т

о производственной практике

Тема задания: **Разработка информационно-аналитической системы, предназначенной для работы с погодными данными**

Обучающийся: Гущин Михаил Олегович, М3309

Место прохождения практики: *ИТМО*

Руководитель практики от профильной организации: Повышев Владислав Вячеславович, доцент (квалификационная категория "доцент практики")

Руководитель практики от университета: Николай Владимирович Андреев, ассистент (квалификационная категория "ассистент")

Дата **26.06.2026**

Санкт-Петербург
2026

Постановка задачи

Цель работы состояла в разработке программного продукта для работы с историческими метеорологическими данными. Система должна загружать погодные данные из открытого источника, сохранять их в базе данных, выполнять аналитические расчеты, находить аномальные значения, строить простой прогноз и формировать итоговый отчет.

Основной смысл проекта - показать полный путь данных: от получения из внешнего API до сохранения в PostgreSQL, расчета показателей, построения графиков и подготовки PDF-отчета.

Пользовательский интерфейс должен быть реализован как консольный запуск с параметрами командной строки. Пользователь указывает города, страну, период анализа и путь к PDF-файлу. После запуска программа сама загружает недостающие данные, анализирует их и создает отчет.

В результате выполнения пользователь получает PDF-файл с аналитическим отчетом по тем параметрам, которые были переданы в командной строке. Этот аналитический отчет системы содержит погодную сводку, графики, аномалии, прогноз и итоговый вывод.

В рамках проекта были выбраны следующие погодные показатели: средняя, минимальная и максимальная температура воздуха, влажность, атмосферное давление, средняя и максимальная скорость ветра, количество осадков, облачность, дата наблюдения, город, страна и координаты.

Проектирование программного продукта

1. описание архитектуры системы;

Система спроектирована как консольное приложение с разделением на четыре проекта:

WeatherAnalytics.Cli - точка входа, разбор параметров и запуск полного сценария;

WeatherAnalytics.Core - бизнес-логика, расчеты, аномалии и прогноз;

WeatherAnalytics.Infrastructure - PostgreSQL, Open-Meteo, загрузка и сохранение данных;

WeatherAnalytics.Reporting - графики и генерация PDF.

2. описание основных компонентов;

Модуль WeatherAnalytics.Cli отвечает за пользовательский запуск. Он проверяет обязательные параметры, формирует ReportRequest, создает зависимости и последовательно вызывает загрузчик данных, построитель отчета, построитель графиков, PDF-генератор и хранилище результатов. На вход он получает уже подготовленные доменные модели, например WeatherDay, а на выходе возвращает структурированный результат WeatherReportData.

Модуль WeatherAnalytics.Core - центральный расчетный модуль проекта. Именно он отвечает за смысловую часть системы: что считать, как группировать данные, какие аномалии искать и как строить прогноз.

Модуль WeatherAnalytics.Infrastructure отвечает за базу данных и внешний источник данных. В нем находится EF Core контекст, сущности базы данных, клиент Open-Meteo и загрузчик погодных данных. Этот слой проверяет, какие данные уже есть в PostgreSQL, загружает только недостающие даты и сохраняет новые записи без дублей.

Модуль WeatherAnalytics.Reporting отвечает за визуализацию и PDF. Он получает готовые аналитические данные, строит графики и формирует PDF отчет.

Модуль WeatherAnalytics.Tests содержит unit-тесты для чистой бизнес-логики.

3. выбор технологий;

C# и .NET 9 используются как основной стек разработки.

PostgreSQL используется как реляционная база данных.

EF Core используется для доступа к базе данных.

Open-Meteo используется как источник данных.

ScottPlot используется для построения графиков.

QuestPDF используется для формирования PDF-отчета.

xUnit используется для unit-тестов.

Docker Compose используется для запуск PostgreSQL

4. проектирование структуры данных;

Структура данных приложения разделена на несколько уровней.

Входные данные пользователя представлены моделью ReportRequest. Она содержит список городов, страну, дату начала периода, дату конца периода, путь к PDF-файлу и порог осадков.

Основная погодная запись представлена моделью WeatherDay. Это дневное наблюдение по одному городу за одну дату. В нем есть температура, влажность, давление, ветер, осадки, облачность и код погоды.

Город для загрузки данных представлен моделью LocationInfo. Она содержит ID города в БД, название, страну, широту, долготу и часовой пояс. Эта модель нужна, чтобы передать Open-Meteo координаты без использования EF-сущности в доменном коде.

Итоговая структура отчета - WeatherReportData. В ней собраны исходные дневные данные, температурные сводки, месячные и сезонные показатели, сравнение городов, тренды, экстремальные дни, аномалии, прогноз и замечания по данным.

5. проектирование интерфейсов;

Пользовательский интерфейс в проекте - это консольный интерфейс. Он состоит из набора параметров командной строки. Обязательные параметры: --cities, --country, --from, --to, --output. Дополнительный параметр --rain-threshold задает порог осадков для аналитического показателя "количество дней с осадками выше порога". Также есть --help, который выводит пример запуска.

6. UML-диаграммы;

Концептуальная диаграмма классов

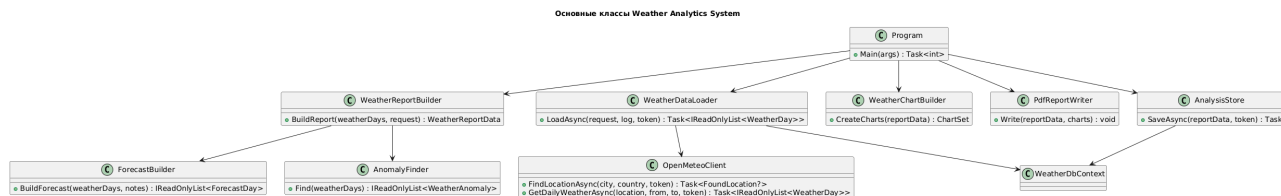
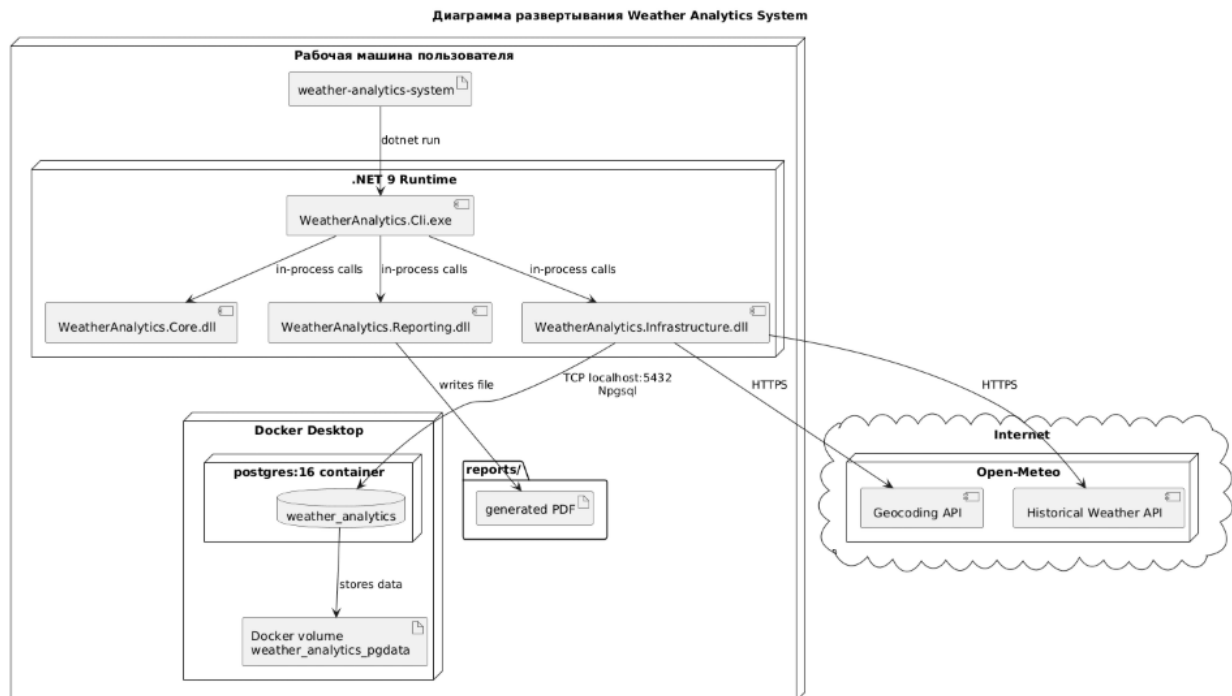
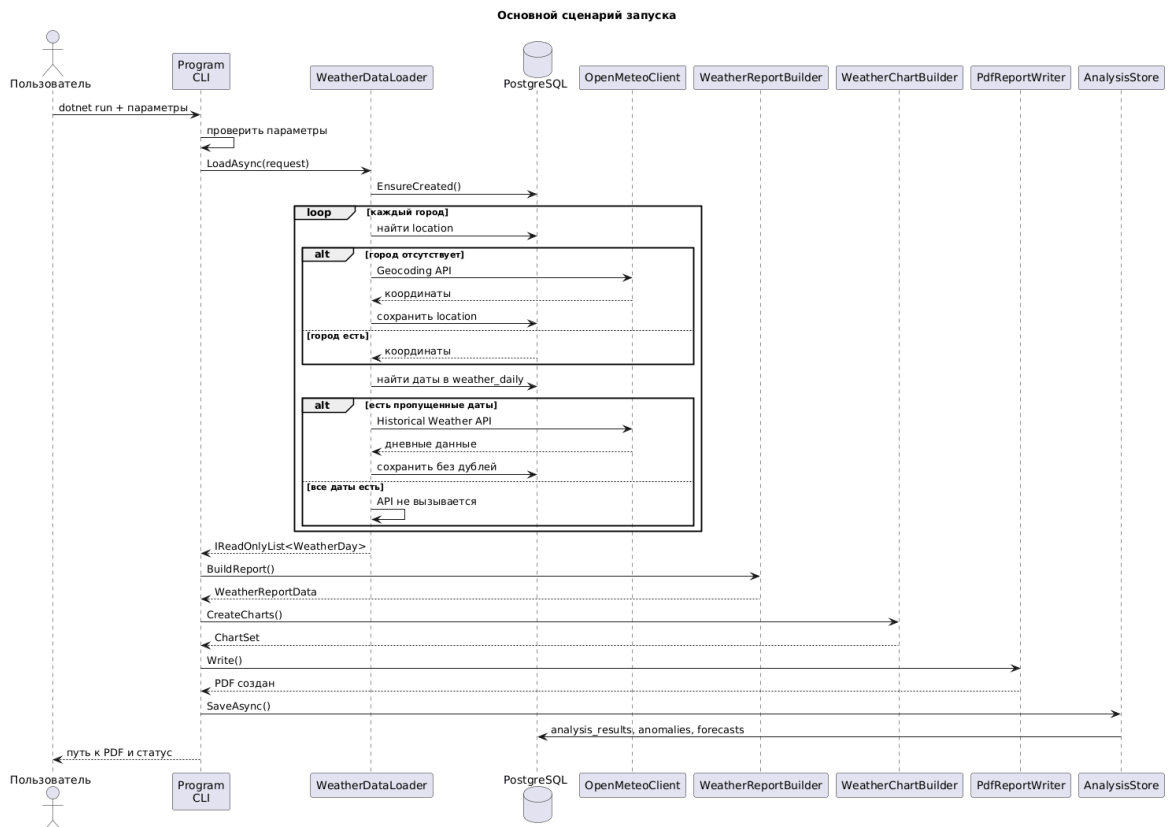


Диаграмма развертывания



7. схемы взаимодействия компонентов;



Система работает в один основной запуск. Пользователь не запускает отдельно команду загрузки и отдельно команду отчета. Приложение само выполняет все этапы:

- получает параметры командной строки;
- проверяет параметры;
- подключается к PostgreSQL;
- проверяет наличие городов в таблице locations;
- если города нет, ищет его через Open-Meteo Geocoding API;
- проверяет наличие дневных погодных записей в weather_daily;
- загружает только недостающие даты;
- сохраняет новые наблюдения;
- читает полный набор данных из БД;
- передает данные в WeatherAnalytics.Core;
- получает структурированный аналитический результат;
- строит графики;
- формирует PDF;
- сохраняет результаты аналитики в отдельные таблицы.

8. описание алгоритмов.

На этапе проектирования были выделены следующие группы алгоритмов:

- агрегация дневных погодных данных по месяцам, сезонам и годам;
- анализ общего количества осадков и количества дождливых дней;
- сравнение городов по температуре, осадкам, ветру и облачности;

- поиск экстремальных дней;
- поиск температурных аномалий через z-score;
- поиск резких температурных изменений через 95-й перцентиль;
- поиск сильных осадков и сильного ветра через 95-й перцентиль;
- поиск сухих периодов;
- прогноз средней температуры на 7 дней по скользящему среднему и линейному тренду.

Проектирование базы данных

1. концептуальная модель данных;

В предметной области есть несколько основных сущностей:

- город;
- дневное погодное наблюдение;
- запуск загрузки данных;
- ошибка загрузки;
- запуск анализа;
- сохраненный результат аналитики;
- найденная аномалия;
- прогнозный день.

Связи между сущностями:

один город имеет много дневных погодных наблюдений;

один запуск загрузки может иметь много ошибок загрузки;

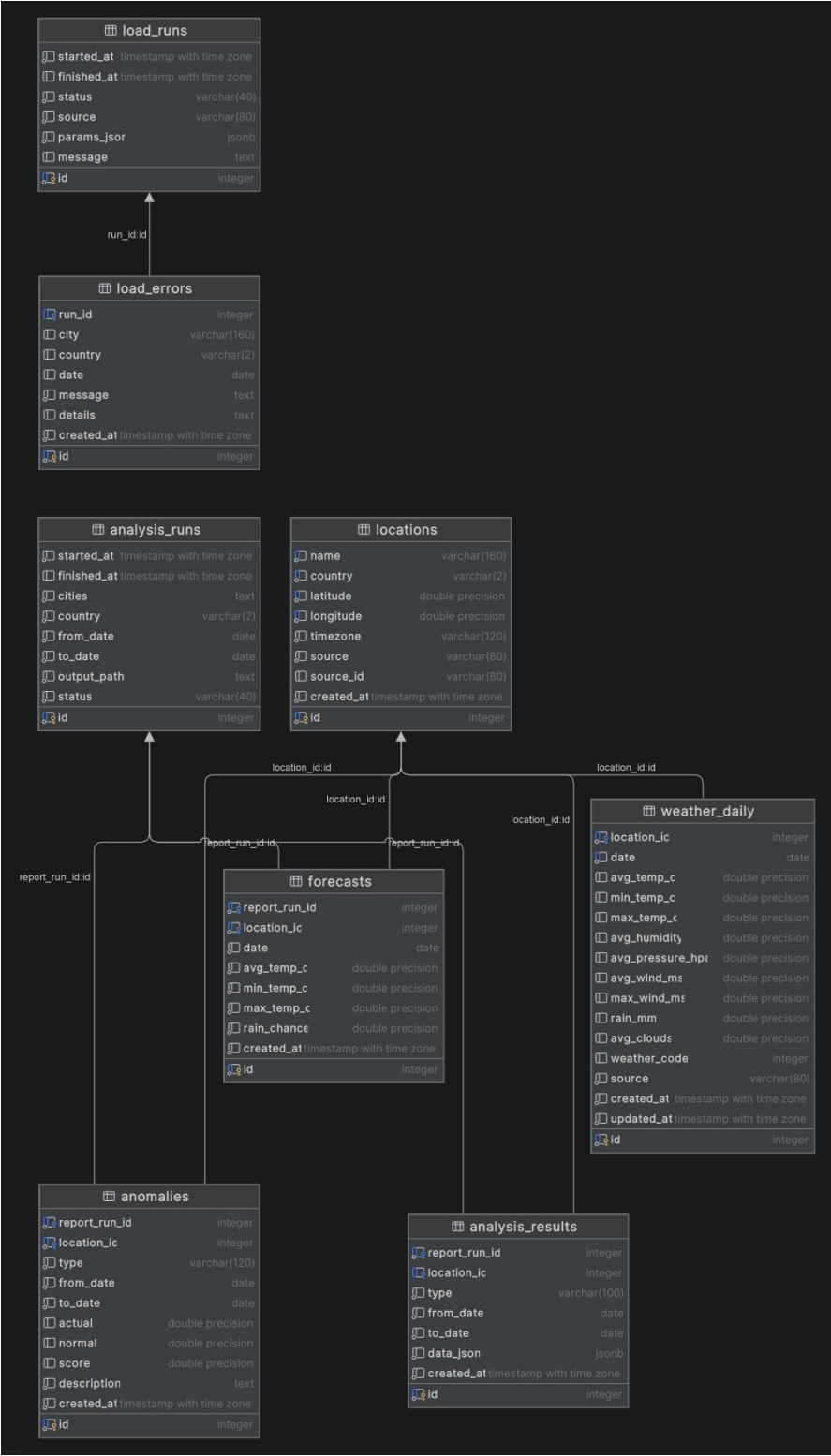
один аналитический запуск имеет много агрегированных результатов;

один аналитический запуск имеет много найденных аномалий;

один аналитический запуск имеет много прогнозных строк;

один город может быть связан со многими результатами анализа, аномалиями и прогнозами.

2. ER-модель;



3. логическая модель данных;

locations

Таблица хранит города и координаты, найденные через Open-Meteo.

Поле	Смысл
id	первичный ключ города
name	название города

country	двухбуквенный код страны
latitude	широта
longitude	долгота
timezone	часовой пояс
source	источник данных, сейчас OpenMeteo
source_id	идентификатор города во внешнем источнике
created_at	время добавления города

Уникальность задана по name, country, latitude, longitude. Это защищает от повторного сохранения одного и того же города с теми же координатами.

weather_daily

Таблица хранит дневные погодные наблюдения. Это основная таблица фактических данных.

Поле	Смысл
id	первичный ключ наблюдения
location_id	внешний ключ на город
date	дата дневного наблюдения
avg_temp_c	средняя температура, градусы Цельсия
min_temp_c	минимальная температура
max_temp_c	максимальная температура
avg_humidity	средняя влажность
avg_pressure_hpa	среднее давление
avg_wind_ms	средняя скорость ветра
max_wind_ms	максимальная скорость ветра
rain_mm	количество осадков
avg_clouds	средняя облачность
weather_code	код погодного состояния
source	источник данных
created_at	время создания записи
updated_at	время обновления записи

Главное ограничение: уникальность пары location_id + date. Это означает, что для одного города за одну дату может быть только одна дневная запись.

load_runs

Таблица хранит журнал запусков загрузки.

Поле	Смысл
id	первичный ключ запуска загрузки
started_at	время начала
finished_at	время окончания
status	running, completed или failed
source	источник данных
params_json	параметры запуска в JSON
message	итоговое сообщение

Эта таблица нужна, чтобы видеть историю ETL-запусков и понимать, успешна ли была загрузка.

load_errors

Таблица хранит ошибки загрузки и проверки данных.

Поле	Смысл
id	первичный ключ ошибки
run_id	внешний ключ на load_runs
city	город, к которому относится ошибка
country	страна
date	дата, если ошибка относится к конкретному дню
message	короткое описание ошибки
details	подробности
created_at	время записи ошибки

analysis_runs

Таблица хранит каждый запуск аналитического отчета.

Поле	Смысл
id	первичный ключ аналитического запуска
started_at	время начала
finished_at	время окончания
cities	города запуска строкой
country	страна
from_date	начало периода

to_date	конец периода
output_path	путь к PDF
status	статус аналитического запуска

Эта таблица отделяет конкретный запуск отчета от исходных погодных данных. Например, один и тот же набор weather_daily может использоваться в разных отчетах с разными городами, периодами и порогами.

analysis_results

Таблица хранит агрегированные аналитические результаты в JSON.

Поле	Смысл
id	первичный ключ результата
report_run_id	внешний ключ на analysis_runs
location_id	внешний ключ на город, может быть null
type	тип результата
from_date	начало периода
to_date	конец периода
data_json	результат в формате JSON
created_at	время сохранения

Сюда сохраняются такие блоки, как temperature_summary, monthly_stats, seasonal_stats, rain_summary, city_comparison, yearly_stats, long_term_trends.

Именно так аналитические результаты сохраняются отдельно: они не добавляются в weather_daily, а пишутся в отдельную таблицу analysis_results, привязанную к конкретной строке analysis_runs. Исходные данные остаются исходными наблюдениями, а рассчитанные показатели хранятся как результаты отчета.

anomalies

Таблица хранит найденные аномалии.

Поле	Смысл
id	первичный ключ аномалии
report_run_id	внешний ключ на аналитический запуск
location_id	внешний ключ на город
type	тип аномалии
from_date	начало даты или периода
to_date	конец даты или периода

actual	фактическое значение
normal	норма или порог
score	сила отклонения
description	текстовое объяснение
created_at	время сохранения

Аномалии вынесены в отдельную таблицу, потому что это не просто агрегированный JSON-блок. По ним удобно фильтровать: по городу, типу, дате, силе отклонения.

forecasts

Таблица хранит прогнозные значения.

Поле	Смысл
id	первичный ключ прогнозной строки
report_run_id	внешний ключ на аналитический запуск
location_id	внешний ключ на город
date	прогнозная дата
avg_temp_c	прогноз средней температуры
min_temp_c	прогноз минимума
max_temp_c	прогноз максимума
rain_chance	вероятность осадков
created_at	время сохранения

Прогнозы вынесены отдельно, потому что это не исторические наблюдения. Их нельзя сохранять в `weather_daily`, иначе исторические факты и прогнозные значения смешаются.

4. описание таблиц и связей;

Таблица `locations` хранит города и их географические координаты. В ней сохраняются название города, страна, широта, долгота, часовой пояс, источник данных и внешний идентификатор города из Open-Meteo. Эта таблица нужна как справочник городов. Если город уже есть в `locations`, приложение не обращается повторно к Geocoding API, а берет координаты из базы данных.

Таблица `weather_daily` хранит дневные погодные наблюдения. Каждая строка относится к одному городу и одной дате. Связь с городом сделана через поле `location_id`, которое ссылается на `locations.id`. Это связь один ко многим: один город из `locations` может иметь много строк в `weather_daily`, потому что для каждого дня создается отдельное

наблюдение. В таблице есть уникальное ограничение по паре `location_id` и `date`, поэтому для одного города за одну дату нельзя сохранить две погодные записи.

Таблица `load_runs` хранит историю запусков загрузки данных. Когда приложение начинает загрузку, оно создает строку со статусом `running`. После успешной загрузки статус меняется на `completed`, а при ошибке - на `failed`. В `params_json` сохраняются параметры запуска, чтобы потом можно было понять, какие города и даты загружались.

Таблица `load_errors` хранит ошибки загрузки. Она связана с `load_runs` через поле `run_id`. Это связь один ко многим: один запуск загрузки может иметь много ошибок. Например, в одном запуске может не найтись один город и одновременно может возникнуть ошибка по другой дате. При удалении записи из `load_runs` ссылка в `load_errors` может стать пустой, потому что используется поведение `SetNull`.

Таблица `analysis_runs` хранит историю запусков аналитического отчета. В ней фиксируются города, страна, период, путь к PDF-файлу и статус. Эта таблица отделяет сам факт запуска отчета от исходных погодных данных. Один и тот же набор исторических данных может использоваться в разных аналитических запусках.

Таблица `analysis_results` хранит агрегированные результаты анализа. Она связана с `analysis_runs` через `report_run_id`: один аналитический запуск может иметь много строк результатов. Например, отдельно сохраняются `temperature_summary`, `monthly_stats`, `seasonal_stats`, `rain_summary`, `city_comparison`, `yearly_stats` и `long_term_trends`. Поле `location_id` здесь необязательное, потому что часть результатов относится ко всему отчету, а часть может относиться к конкретному городу.

Таблица `anomalies` хранит найденные погодные аномалии. Она связана с `analysis_runs` через `report_run_id` и с `locations` через `location_id`. Это значит, что каждая аномалия относится одновременно к конкретному запуску отчета и к конкретному городу. В таблице сохраняется тип аномалии, дата или период, фактическое значение, норма или порог, сила отклонения и текстовое описание.

Таблица `forecasts` хранит прогнозные значения. Она связана с `analysis_runs` через `report_run_id` и с `locations` через `location_id`. Каждая строка - это прогноз на один день для одного города. Прогнозы не сохраняются в `weather_daily`, потому что `weather_daily` содержит исторические факты, а `forecasts` содержит расчетные будущие значения.

Итоговая логика связей такая: `locations` и `weather_daily` хранят исходные погодные данные, `load_runs` и `load_errors` хранят историю загрузки, а `analysis_runs`, `analysis_results`, `anomalies` и `forecasts` хранят результаты конкретных аналитических запусков.

5. описание ограничений целостности.

В базе данных заданы следующие важные ограничения:

- первичные ключи во всех таблицах;
- внешний ключ `weather_daily.location_id` -> `locations.id`;
- внешний ключ `load_errors.run_id` -> `load_runs.id`;
- внешние ключи результатов анализа к `analysis_runs`;

- уникальность locations по набору name, country, latitude, longitude;
- уникальность weather_daily по паре location_id, date;
- индексы по weather_daily.date и weather_daily.location_id.
- если удалить город, его weather_daily удаляются каскадно;
- если удалить аналитический запуск, связанные analysis_results, anomalies и forecasts удаляются каскадно;
- если удалить load_runs, у load_errors ссылка может стать null, потому что ошибка сама по себе может быть полезна для диагностики.

Особенно важна уникальность weather_daily(location_id, date). Благодаря ей повторный запуск за тот же период не создает дубликаты дневных наблюдений.

Проектирование процессов обработки данных

1. описание источников данных;

Источник данных в проекте - Open-Meteo. Для поиска города используется Open-Meteo Geocoding API. Для загрузки исторической погоды используется Open-Meteo Historical Weather API.

2. описание процесса загрузки данных;

Первый шаг - разбор параметров командной строки. Программа проверяет, что список городов не пустой, страна задана двухбуквенным кодом, дата начала не позже даты конца, путь вывода заканчивается на .pdf, а порог осадков не является отрицательным числом.

Второй шаг - подготовка базы данных. Программа создает схему PostgreSQL через EF Core, если она еще не создана. Затем создается запись в журнале загрузки load_runs со статусом running.

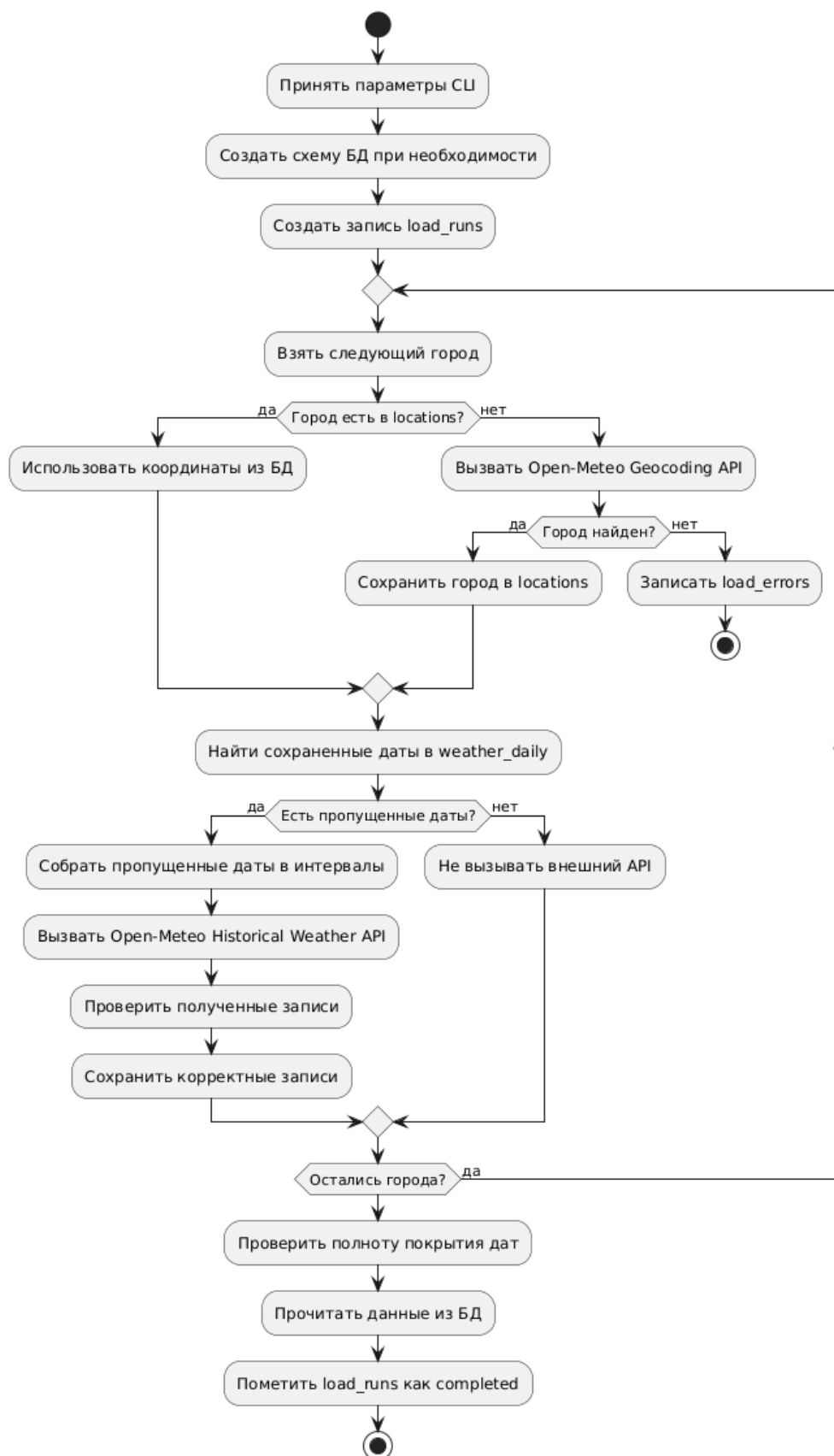
Третий шаг - обработка городов. Для каждого города программа сначала ищет запись в locations. Если город уже есть в базе, используются сохраненные координаты. Если города нет, программа обращается к Open-Meteo Geocoding API и выбирает результат с нужным кодом страны. Если город не найден, запуск завершается ошибкой, PDF не создается, а причина записывается в load_errors.

Четвертый шаг - проверка уже сохраненных погодных данных. Для каждого найденного города программа строит список всех дат от --from до --to. Затем она читает из weather_daily даты, которые уже сохранены. На основе сравнения формируется список отсутствующих дат.

Пятый шаг - загрузка только недостающих интервалов. Если все даты уже есть в базе, программа не вызывает Open-Meteo Historical Weather API. Если часть дат отсутствует,

недостающие даты объединяются в непрерывные интервалы, и API вызывается только для этих интервалов.

ETL-процесс загрузки погодных данных



3. описание процесса преобразования данных;

Данные из Open-Meteo приходят как JSON-массивы по дневным показателям. Клиент OpenMeteoClient преобразует их в список WeatherDay. При этом выполняется сопоставление внешних полей API с внутренними понятными названиями:

- temperature_2m_mean в AvgTempC;
- temperature_2m_min в MinTempC;
- temperature_2m_max в MaxTempC;
- relative_humidity_2m_mean в AvgHumidity;
- pressure_msl_mean в AvgPressureHpa;
- wind_speed_10m_mean в AvgWindMs;
- wind_speed_10m_max в MaxWindMs;
- precipitation_sum в RainMm;
- cloud_cover_mean в AvgClouds;
- weather_code в WeatherCode.

После получения данных загрузчик проверяет корректность:

- дата должна попадать в запрошенный период;
- хотя бы один температурный показатель должен быть заполнен;
- числовые значения не должны быть NaN или бесконечностью;
- осадки и скорость ветра не могут быть отрицательными;
- влажность и облачность должны быть в диапазоне от 0 до 100;
- давление должно быть положительным.

4. описание процесса хранения данных;

Хранение исходных погодных данных

Когда система получает дневные данные из Open-Meteo, она не сразу строит отчет только из ответа API. Сначала данные сохраняются в PostgreSQL. Это нужно для повторного использования и защиты от повторной загрузки.

Последовательность такая:

1. WeatherDataLoader получает список дат, которые уже есть в weather_daily.
2. По запрошенному периоду строится полный список ожидаемых дат.
3. Из полного списка вычитаются даты, которые уже есть.
4. Остаются только пропущенные даты.
5. Пропущенные даты объединяются в интервалы.
6. Для интервалов выполняется запрос к Open-Meteo.
7. Каждая запись проходит проверку.

8. Корректная запись преобразуется в WeatherDailyEntity.
9. Запись добавляется в weather_daily.

Уникальный индекс location_id + date дополнительно защищает от дублей на уровне БД. Даже если логика приложения ошибочно попытается записать повторную дату, база данных не должна разрешить такой дубль.

После построения WeatherReportData вызывается AnalysisStore.SaveAsync.

Сначала создается запись в analysis_runs:

cities = список городов

country = страна

from_date = начало периода

to_date = конец периода

output_path = путь к PDF

status = completed

Затем сохраняются три группы результатов.

Первая группа - агрегированные результаты в analysis_results. Каждый тип результата сохраняется отдельной строкой:

temperature_summary

monthly_stats

seasonal_stats

rain_summary

city_comparison

yearly_stats

long_term_trends

Значение результата сериализуется в JSON и помещается в поле data_json. Это удобно, потому что разные типы аналитики имеют разную структуру. Например, месячная статистика и сравнение городов имеют разные поля, но обе структуры можно сохранить в jsonb.

Вторая группа - аномалии в таблице anomalies. Каждая аномалия сохраняется отдельной строкой, потому что у нее есть четкие поля: город, период, тип, факт, норма, score и описание.

Третья группа - прогнозы в таблице forecasts. Каждый прогнозный день каждого города сохраняется отдельной строкой.

5. описание аналитической обработки данных.

После загрузки данные читаются из БД и передаются в WeatherReportBuilder. Он строит:

- температурную сводку по каждому городу;
- месячную статистику;
- сезонную статистику;
- сводку по осадкам;
- сравнение городов;
- годовую статистику;
- долгосрочный тренд;
- списки самых холодных, теплых и дождливых дней;
- список аномалий;
- прогноз на 7 дней;
- замечания о неполных данных.

После этого WeatherChartBuilder строит графики, а PdfReportWriter помещает таблицы, графики, методику и итоговый вывод в PDF.

Реализация программного продукта

1. структура проекта;

```
WeatherAnalyticsSystem.sln
src/
  WeatherAnalytics.Cli/
  WeatherAnalytics.Core/
  WeatherAnalytics.Infrastructure/
  WeatherAnalytics.Reporting/
tests/
  WeatherAnalytics.Tests/
docker-compose.yml
```

2. описание модулей;

WeatherAnalytics.Cli содержит Program.cs. В нем выполняется:

- проверка --help;
- разбор аргументов;
- проверка формата дат;
- проверка ISO-кода страны;
- проверка расширения PDF-файла;

- сборка строки подключения к PostgreSQL;
- создание всех сервисов;
- запуск полного сценария.

WeatherAnalytics.Core содержит:

- модели отчета и погодных данных;
- правила расчетов WeatherRules;
- построитель отчета WeatherReportBuilder;
- поиск аномалий AnomalyFinder;
- прогноз ForecastBuilder;
- вспомогательные расширения для чисел.

WeatherAnalytics.Infrastructure содержит:

- EF Core контекст WeatherDbContext;
- сущности таблиц;
- WeatherDataLoader;
- OpenMeteoClient;
- AnalysisStore;
- исключение WeatherDataException;
- настройки источника данных.

WeatherAnalytics.Reporting содержит:

- WeatherChartBuilder для PNG-графиков;
- PdfReportWriter для PDF;
- ChartSet для передачи путей к графикам.

WeatherAnalytics.Tests содержит unit-тесты для расчетной части.

3. описание классов;

Модуль	Класс или record	Зачем нужен
WeatherAnalytics.Cli	Program	Точка входа. Разбирает параметры CLI, создает зависимости, запускает загрузку, аналитику, графики, PDF и сохранение результатов.
WeatherAnalytics.Core	WeatherReportBuilder	Главный построитель аналитического результата. Собирает сводки, агрегации, сравнения, экстремумы, аномалии

		и прогноз в WeatherReportData.
WeatherAnalytics.Core	AnomalyFinder	Ищет температурные аномалии, резкие изменения температуры, сильные осадки, сильный ветер и сухие периоды.
WeatherAnalytics.Core	ForecastBuilder	Строит простой прогноз на 7 дней по последним 30 дням истории.
WeatherAnalytics.Core	WeatherRules	Хранит константы алгоритмов: пороги аномалий, длину прогноза, длину сухого периода и другие правила.
WeatherAnalytics.Core	NumberExtensions	Вспомогательные методы для расчетов с nullable-числами: среднее, минимум, сумма без неправильного превращения отсутствующих значений в ноль.
WeatherAnalytics.Core	ReportRequest	Параметры пользовательского запуска: города, страна, период, путь к PDF, порог осадков.
WeatherAnalytics.Core	WeatherDay	Доменная модель одного дневного погодного наблюдения. Используется расчетными алгоритмами.
WeatherAnalytics.Core	WeatherReportData	Итоговый контейнер всех результатов аналитики. Передается в графики, PDF и сохранение результатов.
WeatherAnalytics.Core	TemperatureSummary	Сводка температуры по городу: средняя, минимум, максимум, даты экстремумов, число дней.
WeatherAnalytics.Core	MonthlyWeatherStats	Месячная статистика по

	at	городу: температуры, осадки, дождливые дни.
WeatherAnalytics.Core	SeasonalWeatherStat at	Сезонная статистика по городу: зима, весна, лето, осень.
WeatherAnalytics.Core	RainSummary	Сводка по осадкам: сумма, среднее, дождливые дни, дни выше порога, самый дождливый день.
WeatherAnalytics.Core	CityComparison	Сравнительные показатели города для таблиц и графиков сравнения.
WeatherAnalytics.Core	YearlyWeatherStat	Годовые значения средней температуры и осадков. Используется для долгосрочного тренда.
WeatherAnalytics.Core	LongTermTrend	Результат анализа долгосрочного изменения температуры и осадков.
WeatherAnalytics.Core	ExtremeDay	Строка для экстремального дня: самый холодный, теплый или дождливый день.
WeatherAnalytics.Core	WeatherAnomaly	Найденная аномалия с типом, датой/периодом, фактом, нормой, score и описанием.
WeatherAnalytics.Core	ForecastDay	Прогноз на один день для одного города.
WeatherAnalytics.Core	LocationInfo	Краткая информация о городе для запроса к Open-Meteo: id, название, страна, координаты, timezone.
WeatherAnalytics.Infra structure	WeatherDataLoader	Выполняет ETL: создает схему, ищет города, догружает пропущенные даты, валидирует данные, пишет load_runs и

		load_errors.
WeatherAnalytics.Infra structure	AnalysisStore	Сохраняет результаты отчета в analysis_runs, analysis_results, anomalies, forecasts.
WeatherAnalytics.Infra structure	WeatherDataException	Исключение для ошибок данных и загрузки, которые нужно показать пользователю понятным сообщением.
WeatherAnalytics.Infra structure	WeatherSources	Хранит имя источника данных OpenMeteo.
WeatherAnalytics.Infra structure.Data	WeatherDb	Создает WeatherDbContext по строке подключения.
WeatherAnalytics.Infra structure.Data	WeatherDbContext	EF Core контекст. Описывает таблицы, ключи, индексы, внешние ключи и ограничения PostgreSQL.
WeatherAnalytics.Infra structure.Data	LocationEntity	EF-сущность таблицы locations.
WeatherAnalytics.Infra structure.Data	WeatherDailyEntity	EF-сущность таблицы weather_daily.
WeatherAnalytics.Infra structure.Data	LoadRunEntity	EF-сущность таблицы load_runs.
WeatherAnalytics.Infra structure.Data	LoadErrorEntity	EF-сущность таблицы load_errors.
WeatherAnalytics.Infra structure.Data	AnalysisRunEntity	EF-сущность таблицы analysis_runs.
WeatherAnalytics.Infra structure.Data	AnalysisResultEntity	EF-сущность таблицы analysis_results.
WeatherAnalytics.Infra structure.Data	AnomalyEntity	EF-сущность таблицы anomalies.
WeatherAnalytics.Infra structure.Data	ForecastEntity	EF-сущность таблицы forecasts.
WeatherAnalytics.Infra	OpenMeteoClient	HTTP-клиент для Open-

structure.OpenMeteo		Meteo Geocoding API и Historical Weather API.
WeatherAnalytics.Infra structure.OpenMeteo	OpenMeteoOptions	Настройки Open-Meteo: базовые URL, формат, timezone, единицы ветра, список daily-показателей.
WeatherAnalytics.Infra structure.OpenMeteo	FoundLocation	Результат поиска города через Geocoding API.
WeatherAnalytics.Repo rting	WeatherChartBuild er	Строит PNG-графики температуры, осадков, сравнения городов и прогноза.
WeatherAnalytics.Repo rting	PdfReportWriter	Формирует аналитический PDF-отчет через QuestPDF.
WeatherAnalytics.Repo rting	ChartSet	Хранит пути к сгенерированным графикам.
WeatherAnalytics.Tests	AnomalyAndForec astTests	Unit-тесты для поиска аномалий и прогноза.

4. описание библиотек и фреймворков;

Библиотека или технология	Где используется	Зачем нужна
.NET 9	все проекты	Платформа выполнения приложения. Дает консольное приложение, async/await, HttpClient, работу с датами, коллекциями и файлами.
Entity Framework Core	WeatherAnalytics .Infrastructure	ORM для работы с PostgreSQL. Позволяет описывать таблицы через C#-классы и выполнять запросы через LINQ.
Npgsql.Entity FrameworkCore.Post greSQL	WeatherAnalytics .Infrastructure	Провайдер EF Core для PostgreSQL. Без него EF Core не смог бы подключаться именно к PostgreSQL.
PostgreSQL 16	база данных	Хранит города, дневные погодные данные, журналы загрузки, результаты аналитики, аномалии и прогнозы.

Docker Compose	запуск БД	Позволяет поднять PostgreSQL одной командой <code>docker compose up -d</code> , без ручной установки сервера БД.
System.Net.Http.HttpClient	OpenMeteoClient	Выполняет HTTP-запросы к Open-Meteo.
System.Net.Http.Json	OpenMeteoClient	Позволяет читать JSON-ответы API сразу в C#-объекты.
System.Text.Json	WeatherDataLoader, AnalysisStore, DTO Open-Meteo	Сериализует параметры запуска и аналитические результаты в JSON, читает JSON-ответы API.
QuestPDF	PdfReportWriter	Формирует PDF-отчет: страницы A4, таблицы, текст, изображения графиков.
ScottPlot	WeatherChartBuilder	Создает PNG-графики, которые затем вставляются в PDF.
xUnit	WeatherAnalytics.Tests	Фреймворк unit-тестирования. Используется для проверки алгоритмов Core.

5. особенности реализации;

Первая особенность - один основной запуск. Пользователь не обязан отдельно загружать данные, а затем отдельно строить отчет. Приложение само определяет, какие данные уже есть, что нужно загрузить, и после этого строит PDF.

Вторая особенность - загрузка только пропущенных дат. Если пользователь повторяет запуск за тот же период, система не обращается к Historical Weather API для уже сохраненных дневных данных. Это экономит время и уменьшает зависимость от внешнего сервиса.

Третья особенность - отдельная обработка случаев, когда данных недостаточно. Например, долгосрочный тренд не считается, если меньше двух лет данных. Прогноз не строится, если меньше 30 дней истории.

6. примеры ключевых фрагментов кода.

Основной сценарий запуска

```
var weatherDays = await loader.LoadAsync(request, Console.Out, CancellationToken.None);
Console.WriteLine($"Дневных наблюдений для анализа: {weatherDays.Count}");
```

```
var reportData = reportBuilder.BuildReport(weatherDays, request);
var chartSet = chartBuilder.CreateCharts(reportData);
```

```
pdf.Write(reportData, chartSet);
await analysisStore.SaveAsync(reportData, CancellationToken.None);
```

Этот фрагмент показывает главный сценарий системы: загрузить данные, построить аналитику, построить графики, создать PDF и сохранить результаты.

Проверка параметров CLI

```
var cities = Required(values, "--cities")
    .Split(',', StringSplitOptions.RemoveEmptyEntries | StringSplitOptions.TrimEntries)
    .Distinct(StringComparer.OrdinalIgnoreCase)
    .ToList();

if (cities.Count == 0)
{
    throw new ArgumentException("Параметр --cities должен содержать хотя бы один город.");
}
```

Этот код защищает систему от пустого списка городов и повторов в параметре --cities.

Создание записи загрузки

```
var loadRun = new LoadRunEntity
{
    StartedAt = DateTimeOffset.UtcNow,
    Status = "running",
    Source = WeatherSources.OpenMeteo,
    ParamsJson = JsonSerializer.Serialize(request)
};

db.LoadRuns.Add(loadRun);
await db.SaveChangesAsync(cancellationToken);
```

Так начинается журналирование ETL-процесса. Даже если дальше произойдет ошибка, система уже знает, что запуск был начат.

Поиск города в БД или Open-Meteo

```
var location = await db.Locations
    .FirstOrDefaultAsync(item =>
        item.Country == cleanCountry
        && item.Name.ToLower() == cleanCity.ToLower(), cancellationToken);

if (location is not null)
{
    await log.WriteLineAsync($"Город {location.Name}: координаты взяты из БД.");
    return location;
}
```



```
var lookup = await openMeteo.FindLocationAsync(cleanCity, cleanCountry, cancellationToken);
```

Этот фрагмент показывает DB-first подход: сначала проверяется локальная БД, и только если города нет, выполняется запрос к Open-Meteo.

Загрузка только пропущенных дат

```
var existingDates = await db.WeatherDaily
    .Where(day => day.LocationId == location.Id && day.Date >= request.From && day.Date <=
request.To)
    .Select(day => day.Date)
    .ToListAsync(cancellationToken);
```

```
var existingDateSet = existingDates.ToHashSet();
var missingDates = EachDate(request.From, request.To)
    .Where(date => !existingDateSet.Contains(date))
    .ToList();
```

```
if (missingDates.Count == 0)
{
    await log.WriteLineAsync($"{location.Name}: все дневные данные уже есть в БД, API не
вызывается.");
    return;
}
```

Это один из главных фрагментов проекта. Он делает повторные запуски экономными и не позволяет каждый раз загружать одни и те же данные.

Проверка корректности дневной записи

```
if (!day.AvgTempC.HasValue && !day.MinTempC.HasValue && !day.MaxTempC.HasValue)
{
    return "Нет основных температурных показателей.";
}
```

```
if (day.RainMm < 0)
{
    return "Количество осадков не может быть отрицательным.";
}
```

```
if (day.AvgHumidity is < 0 or > 100)
{
    return "Влажность должна быть в диапазоне 0-100.";
}
```

Проверка нужна, чтобы в базу не попадали явно некорректные данные.

Уникальность дневных наблюдений

```
entity.HasIndex(item => new { item.LocationId, item.Date }).IsUnique();
```

Этот фрагмент находится в WeatherDbContext. Он гарантирует, что в таблице weather_daily не будет двух записей для одного города и одной даты.

Сборка аналитического отчета

```
var temperatureSummaries = BuildTemperatureSummaries(orderedDays, notes);
var monthlyStats = BuildMonthlyStats(orderedDays);
var seasonalStats = BuildSeasonalStats(orderedDays);
var rainSummaries = BuildRainSummaries(orderedDays, request.RainThresholdMm, notes);
var cityComparisons = BuildCityComparisons(orderedDays);
var anomalies = anomalyFinder.Find(orderedDays);
var forecasts = forecastBuilder.BuildForecast(orderedDays, notes);
```

Этот код показывает, что WeatherReportBuilder собирает отчет из нескольких независимых аналитических блоков.

Температурная аномалия через z-score

```
var monthAvg = monthDays.Average(day => day.AvgTempC!.Value);
var monthStd = StandardDeviation(monthDays.Select(day => day.AvgTempC!.Value));
var z = (day.AvgTempC!.Value - monthAvg) / monthStd;
```

```
if (z >= WeatherRules.TemperatureAnomalyZScore)
{
    // аномально высокая температура
}
else if (z <= -WeatherRules.TemperatureAnomalyZScore)
{
    // аномально низкая температура
}
```

День сравнивается не со всеми днями подряд, а с днями того же месяца. Это делает поиск сезонных температурных отклонений более корректным.

Прогноз средней температуры

```
var avg7 = last7.Average(day => day.AvgTempC!.Value);
var trendPerDay = LinearSlope(last30);

for (var dayNumber = 1; dayNumber <= WeatherRules.ForecastDays; dayNumber++)
{
    var avgTemp = avg7 + trendPerDay * dayNumber;
    result.Add(new ForecastDay(...));
}
```

Формула простая и объяснимая: берется среднее за последние 7 дней и корректируется на тренд последних 30 дней.

Сохранение агрегированных результатов отдельно

```
AddResult(run.Id, null, "temperature_summary", request, reportData.TemperatureSummaries);
AddResult(run.Id, null, "monthly_stats", request, reportData.MonthlyStats);
AddResult(run.Id, null, "seasonal_stats", request, reportData.SeasonalStats);
AddResult(run.Id, null, "rain_summary", request, reportData.RainSummaries);
AddResult(run.Id, null, "city_comparison", request, reportData.CityComparisons);
```

Здесь видно, как аналитика сохраняется отдельно от исходной погоды: не в `weather_daily`, а в `analysis_results` с указанием типа результата.

Сохранение аномалий

```
db.Anomalies.Add(new AnomalyEntity
{
    ReportRunId = run.Id,
    LocationId = anomaly.LocationId,
    Type = anomaly.Type,
    FromDate = anomaly.FromDate,
    ToDate = anomaly.ToDate,
    Actual = anomaly.Actual,
    Normal = anomaly.Normal,
    Score = anomaly.Score,
    Description = anomaly.Description,
    CreatedAt = DateTimeOffset.UtcNow
});
```

Каждая аномалия сохраняется отдельной строкой, потому что по аномалиям удобно делать отдельные запросы.

Агрегация графика для длинных периодов

```
if ((reportData.Request.To.DayNumber - reportData.Request.From.DayNumber) > 365)
{
    foreach (var cityGroup in reportData.MonthlyStats.GroupBy(item => item.City))
    {
        AddDateScatter(plot,
            points.Select(item => new DateOnly(item.Year, item.Month, 1)),
            points.Select(item => item.AvgTempC!.Value),
            cityGroup.Key);
    }
}
```

Если период больше года, график температуры строится по месяцам. Это делает PDF читаемым для больших отчетов.

Реализация алгоритмов

1. описание реализованных алгоритмов;

Агрегация погодных данных

Дневные наблюдения группируются по городу и периоду. Для каждого города рассчитываются:

- средняя температура за период;
- минимальная температура;
- максимальная температура;
- дата самого холодного дня;
- дата самого теплого дня;
- общее число дней с данными.

Месячная статистика строится группировкой по LocationId, городу, году и месяцу.

Сезонная статистика строится по сезонам:

- зима - декабрь, январь, февраль;
- весна - март, апрель, май;
- лето - июнь, июль, август;
- осень - сентябрь, октябрь, ноябрь.

Для зимы декабрь относится к следующему сезонному году. Например, декабрь 2023 года входит в зиму 2024 года. Это сделано потому, что зима как сезон пересекает границу календарного года.

Анализ осадков

Для каждого города рассчитываются:

- общая сумма осадков;
- среднее количество осадков в день;
- количество дождливых дней;
- количество дней выше пользовательского порога --rain-threshold;
- самый дождливый день.

В проекте есть два порога:

- WeatherRules.RainyDayMm = 0.1 мм - системный порог для определения дождливого дня;
- --rain-threshold - пользовательский порог для отдельного аналитического показателя.

Сравнение городов

Сравнение строится только если выбрано два или больше городов. Для каждого города выводятся:

- средняя температура;

- минимальная температура;
- максимальная температура;
- сумма осадков;
- количество дождливых дней;
- средняя скорость ветра;
- средняя облачность.

Если выбран один город, в PDF выводится пояснение, что сравнение не выполнялось. Это корректнее, чем строить пустую сравнительную диаграмму.

Долгосрочный тренд

Долгосрочный тренд рассчитывается по годовой средней температуре. Для города должно быть не меньше двух лет данных. Если период короче, система добавляет замечание, что тренд не рассчитывался.

Тренд считается через наклон линейной регрессии:

$$\text{slope} = \text{sum}((x - \text{avg_x}) * (y - \text{avg_y})) / \text{sum}((x - \text{avg_x})^2)$$

Где:

- x - год;
- y - средняя температура за этот год;
- slope - изменение температуры в градусах Цельсия за год.

Если модуль тренда меньше 0.05 градуса в год, система считает, что заметного изменения нет.

Поиск температурных аномалий

Температурные аномалии ищутся отдельно для каждого города и каждого месяца года. Например, все январские дни сравниваются только с январскими днями этого же города. Это важно, потому что температура в январе и июле имеет разные нормы.

Используется z-score:

$$z = (\text{avg_temp_c} - \text{month_avg_temp_c}) / \text{month_temp_std}$$

Если $z \geq 2$, день считается аномально теплым. Если $z \leq -2$, день считается аномально холодным.

Поиск резких изменений температуры

Для соседних дней одного города считается изменение средней температуры. Если даты идут подряд, вычисляется модуль изменения. Затем по всем таким изменениям определяется 95-й перцентиль. Изменения выше этого порога считаются резкими.

Система различает:

- резкий рост температуры;
- резкое падение температуры.

В результат записываются дата, фактическое изменение, порог и отношение факта к порогу.

Поиск сильных осадков и сильного ветра

Для сильных осадков берутся только дни, где $\text{rain_mm} \geq 0.1$. Затем считается 95-й перцентиль по осадкам. Дни выше порога попадают в список аномалий.

Для сильного ветра используется показатель max_wind_ms . По нему также считается 95-й перцентиль. Если дневное значение выше порога, фиксируется аномально сильный ветер.

Поиск сухих периодов

Сухим считается день, где $\text{rain_mm} < 0.1$. Если таких дней подряд 7 или больше, система создает аномалию "Длительный период без осадков".

Алгоритм учитывает непрерывность дат. Если между записями есть пропуск по датам, сухой период не склеивается через этот пропуск. Если rain_mm отсутствует, день не считается сухим, потому что неизвестное значение нельзя приравнять к нулю.

Прогноз на 7 дней

Для средней температуры используется формула:

$$\text{forecast_temp_c} = \text{avg_7_days} + \text{trend_per_day} * \text{day_number}$$

Где:

- avg_7_days - средняя температура за последние 7 дней;
- trend_per_day - наклон линейной регрессии за последние 30 дней;
- day_number - номер прогнозного дня от 1 до 7.

Минимальная и максимальная прогнозная температура строятся через средний разрыв между средней температурой и минимумом/максимумом за последние 14 дней.

Вероятность осадков считается как доля дождливых дней за последние 14 дней:

$$\text{rain_chance} = \text{rainy_days_last_14} / \text{days_with_rain_data_last_14}$$

Если за последние 14 дней нет данных по осадкам, вероятность осадков не рассчитывается и в отчет добавляется замечание.

2. оптимизации;

В проекте использованы простые оптимизации, которые прямо связаны с задачей:

- данные не загружаются повторно, если они уже есть в weather_daily ;
- пропущенные даты объединяются в интервалы, чтобы не делать отдельный API-запрос на каждый день;
- для отчетов длиннее года температурный график строится по месячным значениям, а не по каждой дневной точке;

3. оценка вычислительной сложности.

Алгоритм	Сложность
Сортировка входных данных	$O(n \log n)$
Температурная сводка	$O(n)$ после группировки, в реализации с LINQ-группировкой примерно $O(n)$
Месячная агрегация	$O(n)$
Сезонная агрегация	$O(n)$
Сводка по осадкам	$O(n)$
Сравнение городов	$O(n)$
Годовая статистика	$O(n)$
Долгосрочный тренд	$O(c * y)$ после годовой агрегации
Поиск экстремальных дней	$O(m \log m)$ на город, в сумме до $O(n \log n)$
Температурные аномалии z-score	$O(n)$ для группировки и прохода по дням
Резкие изменения температуры	$O(m \log m)$ на город из-за сортировки и сортировки значений для перцентиля
Сильные осадки	$O(m \log m)$ на город из-за сортировки для перцентиля
Сильный ветер	$O(m \log m)$ на город
Сухой период	$O(m \log m)$ с учетом сортировки, сам проход $O(m)$
Прогноз на 7 дней	$O(m \log m)$ на город из-за сортировки; расчет после сортировки $O(30 + 7)$
Построение температурного графика	$O(p)$ после подготовки данных
Построение PDF	$O(r + g)$, где r - количество строк таблиц, g - число графиков

Экспериментальная часть

1. используемые данные;

В экспериментальной части были проверены три сценария. Они отличаются количеством городов и длиной периода. Это нужно, чтобы убедиться, что система работает не только на маленьком примере, но и на более крупном отчете.

2. сценарии экспериментов;

1 эксперимент – создание отчета с данными: период 1 месяц, число городов – 1.

2 эксперимент – создание отчета с данными: период 1 год, число городов – 3.

3 эксперимент – создание отчета с данными: период 3 года, число городов – 2.

3. параметры экспериментов.

Эксперимент 1: 1 город, 1 месяц

Параметры:

- город: Moscow;
- период: 2025-01-01 - 2025-01-31;

Эксперимент 2: 3 города, 1 год

Параметры:

- города: Moscow, Saint Petersburg, Sochi;
- период: 2025-01-01 - 2025-12-31;

Эксперимент 2: 3 города, 1 год

Параметры:

- города: Moscow, Saint Petersburg, Sochi;
- период: 2025-01-01 - 2025-12-31;

Тестирование

Тест	Что проверяет
AnomalyFinder_FindsTemperatureAnomalyByZScore	Находит аномально высокую температуру через z-score.
AnomalyFinder_FindsDryPeriodAfterSevenDryDays	Находит сухой период после 7 дней без осадков.
AnomalyFinder_DoesNotJoinDryPeriodAcrossDateGaps	Не склеивает сухой период через пропуски дат.
AnomalyFinder_DoesNotTreatMissingRainAsDry	Не считает null по осадкам сухим днем.

ForecastBuilder_ReturnsSevenForecastDays	Возвращает ровно 7 прогнозных дней.
ForecastBuilder_CalculatesRainChanceFromLastFourteenDays	Считает вероятность осадков по последним 14 дням.
ForecastBuilder_LeavesRainChanceEmptyWhenRecentRainDataIsMissing	Оставляет вероятность осадков пустой, если свежих данных по осадкам нет.

Анализ результатов

1. анализ полученных результатов;

В результате работы был получен программный продукт, который покрывает основные требования задания:

2. загружает данные из открытого источника;
3. хранит данные в базе данных;
4. поддерживает фильтрацию по городам и датам;
5. выполняет агрегацию по дням, месяцам, сезонам и годам;
6. строит аналитические таблицы;
7. выявляет аномалии;
8. строит простой прогноз;
9. визуализирует результаты в графиках;
10. формирует итоговый PDF-отчет.

11. сравнение с ожидаемыми результатами;

Ожидаемый результат по заданию - система для загрузки, хранения, анализа, прогнозирования и визуализации погодных данных. Реализованное приложение соответствует этому результату.

12. оценка эффективности решения;

Решение эффективно для не промышленной системы анализа и прогноза погоды:

- не делает лишних API-запросов;
- хранит данные с уникальным ограничением;
- разделяет загрузку, расчет и отчетность по модулям;
- расчетная часть покрыта unit-тестами;
- PDF формируется автоматически после загрузки и анализа;
- для длинных периодов графики агрегируются, чтобы не становиться нечитаемыми.

13. оценка производительности;

Основные затраты времени:

- обращение к Open-Meteo при первом запуске;

- запись новых данных в PostgreSQL;
- построение PNG-графиков;
- генерация PDF.

Повторные запуски быстрее, потому что уже сохраненные дневные данные читаются из БД.

14. ограничения разработанного решения.

используется один источник данных - Open-Meteo;

нет графического интерфейса, потому что по заданию используется консоль;

миграции EF Core не выделены отдельно, схема создается через EnsureCreated;

Заключение

1. выполненные задачи;

В ходе работы была разработана информационно-аналитическая система Weather Analytics System. Система реализована как консольное приложение на C# и .NET 9. Она принимает параметры командной строки, работает с PostgreSQL, загружает данные из Open-Meteo, выполняет аналитику, ищет аномалии, строит простой прогноз и создает PDF-отчет.

2. полученные результаты;

В проекте реализована архитектура с разделением ответственности. CLI отвечает за запуск, Core - за расчеты, Infrastructure - за базу данных и внешний API, Reporting - за графики и PDF, Tests - за проверку расчетов.

Была спроектирована и реализована база данных для городов, дневных погодных наблюдений, журналов загрузки, результатов аналитики, аномалий и прогнозов. Для дневных наблюдений задано ограничение уникальности по городу и дате, что защищает систему от дублей.

Были реализованы ETL-процесс, аналитические алгоритмы, поиск аномалий, прогноз на 7 дней, построение графиков и генерация PDF. Были выполнены тестовые прогоны на разных городах и периодах.

3. выводы по работе.

Основной результат работы - готовый программный продукт, который можно запустить одной командой и получить аналитический PDF-отчет по выбранным погодным данным.

Список использованных источников

10. QuestPDF Documentation: <<https://www.questpdf.com/>>
11. ScottPlot Documentation: <<https://scottplot.net/>>
12. Open-Meteo Historical Weather API: <<https://open-meteo.com/en/docs/historical-weather-api>>
13. Open-Meteo Geocoding API: <<https://open-meteo.com/en/docs/geocoding-api>>

14. Microsoft .NET Documentation: <<https://learn.microsoft.com/en-us/dotnet/>>
15. Microsoft Entity Framework Core Documentation: <<https://learn.microsoft.com/en-us/ef/core/>>
16. Npgsql Entity Framework Core Provider: <<https://www.npgsql.org/efcore/>>
17. PostgreSQL Documentation: <<https://www.postgresql.org/docs/>>

Ссылки на репозиторий

<https://github.com/Miraclejke/weather-analytics-system>

Приложения

1. UML-диаграммы;

Концептуальная диаграмма классов

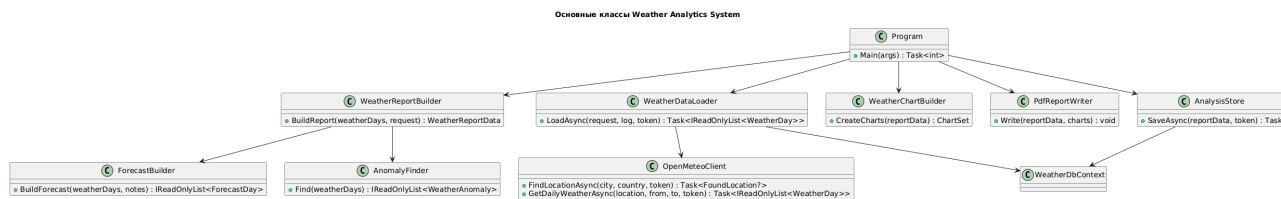


Диаграмма Последовательности

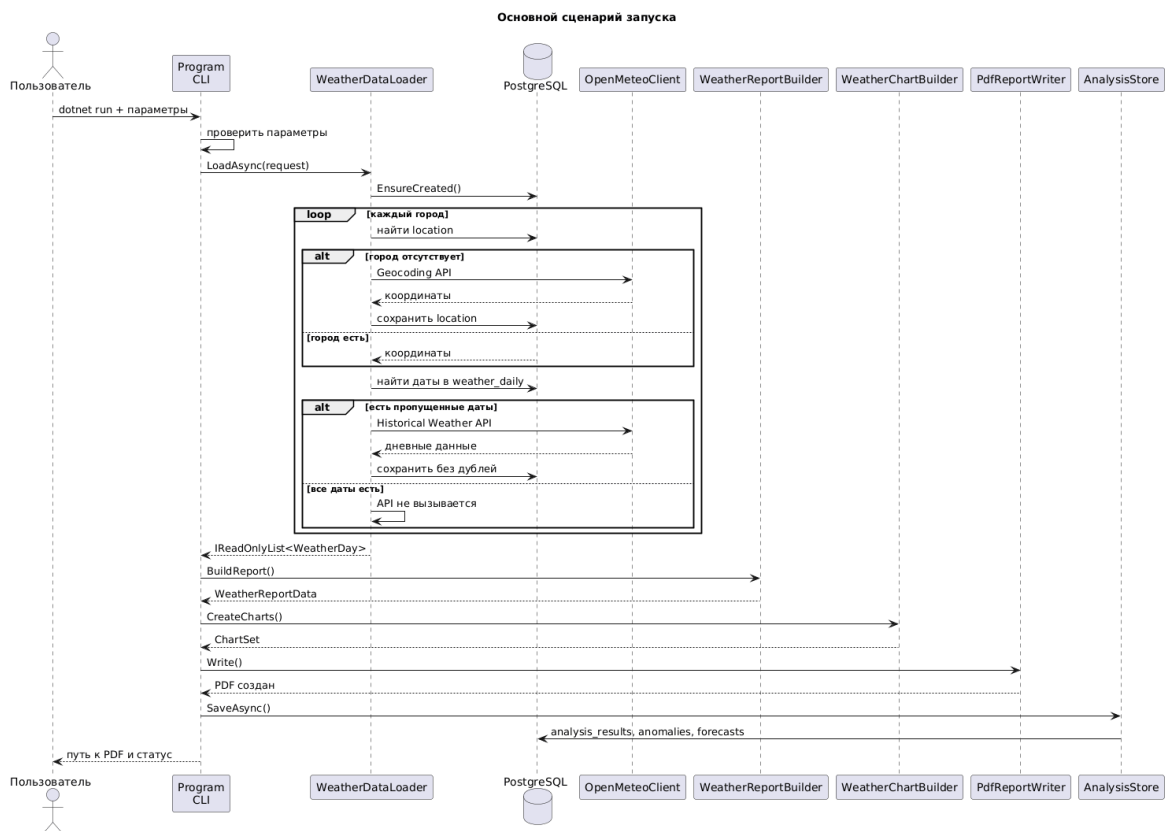
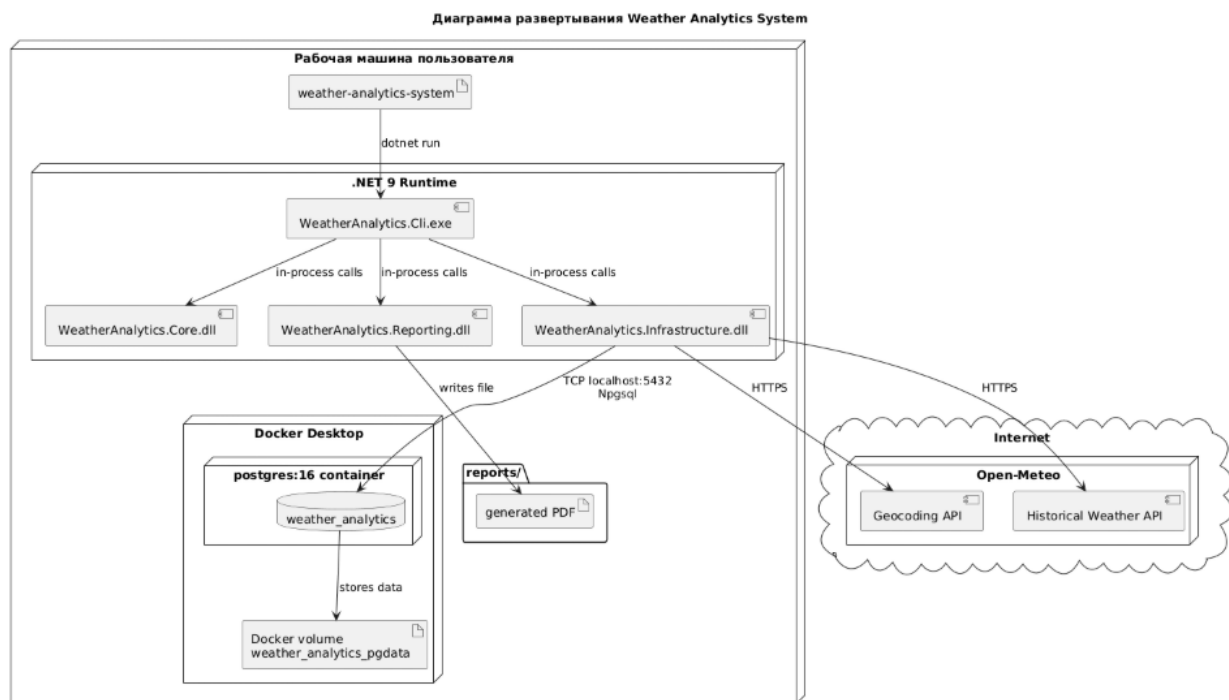
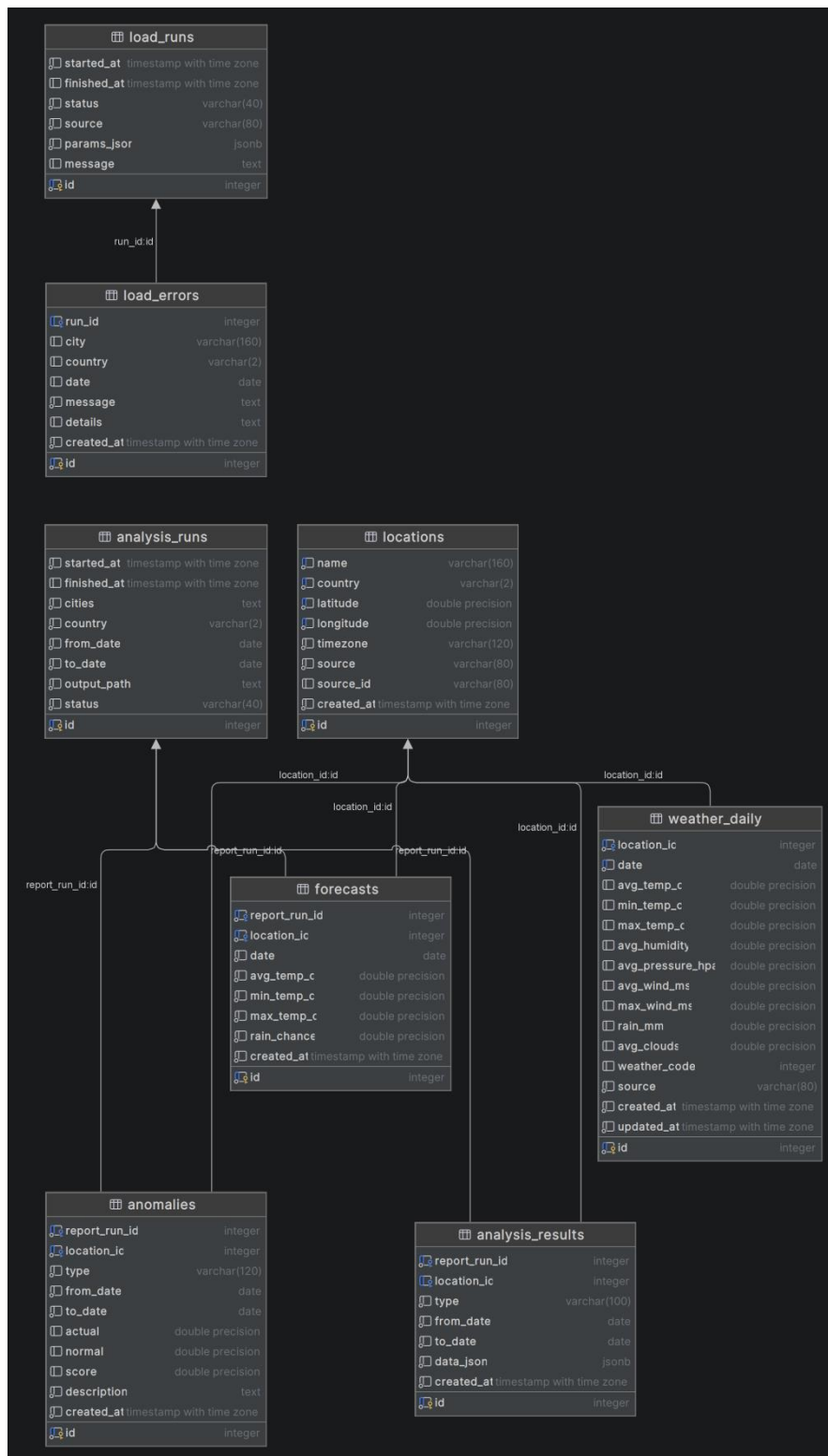


Диаграмма разворачивания



2. ER-модели;



3. SQL-скрипты;

SQL-запрос для проверки дублей

```

select count(*) as weather_rows,
       count(distinct (location_id, date)) as unique_weather_rows
from weather_daily;
  
```

4. результаты экспериментов;

Эксперименты показывают, что система работает на разных объемах данных:

- на малом объеме отчет остается коротким;
- при нескольких городах появляются сравнительные таблицы и графики;
- при многолетнем периоде отчет становится больше, но остается структурированным;
- количество наблюдений совпадает с ожидаемым числом дней и городов;
- PDF создается во всех трех сценариях.

5. инструкции по запуску.

Запустить PostgreSQL:

```
docker compose up -d
```

Собрать проект:

```
dotnet build
```

Выполнить основной запуск:

```
dotnet run --project src/WeatherAnalytics.Cli -- \
```

```
--cities Moscow, Saint-Petersburg \
```

```
--country RU \
```

```
--from 2021-01-01 \
```

```
--to 2025-01-01 \
```

```
--output reports/report.pdf
```